CS 381 Intro to Artificial Intelligence Fall 2026

Project 3
(Due: Nov 1, 2026)

Consider the following variation of *Tic-tac-toe*: The game is played on a 4 by 4 board in which players take turns placing 1 or 2 on each square, just as in Tic-tac-toe. The only change is how we define a win: a player wins if (s)he has captured four squares no two of which are in the same row or in the same column. More formally, a win for a player requires marking a set of four squares $(a_1, b_1), (a_2, b_2), (a_3, b_3), (a_4, b_4)$ where $\{a_1, a_2, a_3, a_4\} = \{1, 2, 3, 4\}$ and $\{b_1, b_2, b_3, b_4\} = \{1, 2, 3, 4\}$.

For example, shown below is a win for player 2 because of 2's in positions (1,1), (2,3), (3,4) and (4,2). (The positions are represented by column number and row number from left to right and from top to bottom. 0 represents an unmarked cell.)

2 1 1 0
1 1 **2** 0
2 0 1 **2**
1 **2** 2 0

The goals of the project are the following: (a) implement an efficient and strong computer player that plays against a human player and create a game interface to play against a human player and (b) use the game-tree expansion to determine the value of a board expanded to a specified depth using a standard static evaluation function. The details on how to accomplish these goals is provided below:

(1) Write a recursive function *Minimax* that takes as input a board as a 4 by 4 vector (such as [2, 1, 0, 0], [0, 1, 0, 0], [0, 0, 0, 0], [0, 0, 2, 0]]) and outputs the value of the board as 1 (if the player 1 can force a win), 0 (if neither player can force a win), -1 (if the player 2 can force a win). Implement a *checkValid* function that checks if a board is valid and returns failure if it is not a valid board. A valid board represents a game that just finished or a is game in progress following the rules of the game. For example, a board with five 2's and three 1's is not a valid board. Another example of an invalid board is one in which both players have a winning marks such as [[**1**, 0, 1, **2**], [0, **1**, **2**, 0], [**2**, 0, 0, **1**], [2, **2**, **1**, 0]]. You are to modify it so that it returns the best move for the current player – breaking ties with a smaller row number and a smaller column number. (For example, suppose there are two winning moves (1, 3) and (2, 4), it should return (1, 3). If the winning moves are (2, 1) and (2, 4), it should output the former. This function should also return the number of nodes expanded and the CPU time.

Procedure Minimax
Input: Position u
Output: Value at root

```
if (leaf(u)) return Eval(u)                    ;; No successor, static evaluation
if (max-node(u)) val ← −∞                      ;; Initialize return value for MAX node
else val ← +∞                                  ;; Initialize return value for MIN node
for each v ∈ Succ(u)                           ;; Traverse successor list
   if (max-node(u)) val ← max{val, Minimax(v)} ;; Recursive call at MAX node
   else val ← min{val, Minimax(v)}             ;; Recursive call at MIN node
return res                                     ;; Return final evaluation
```

Algorithm 12.2

Minimax game tree search.

(2) Write a static evaluation function *eval*(*B*, *p*) (where *p* is 1 or 2) that takes as input a board *B* and determines the value of *B* (for the player) defined as follows. For a non-leaf node, *e*(*B*, *p*) is the number of ways in which the player *p* can win – the number of ways in which the opponent can win.

Shown below is a board *B* for which the value of *eval*(*B*, 1) is 8 – 7 = 1.

1	2	1	2
2	1	0	0
1	0	0	0
2	0	0	0

(3) Using the static evaluation function *eval*(*B*, *p*), write functions *MinimaxCutOff* and *MinimaxAlphaBetaCutOff* that take as input a board and a positive integer *k* as input and performs a search to *k* plies (i.e., to depth *k*), and determines the best move for the root player using Minimax with the tree explored to depth *k* and alpha-beta pruning explored to depth *k*. Also the value at the root node should be returned. Pseudo-code for *MinimaxAlphaBeta* is shown below. You need to modify it so that the output includes not only the value of the current board but also the move that gives this value. Note that the value returned will be for the root player. Thus a positive value returned at the root means the root player has a greater likelihood of winning. (But this is not guaranteed since the static evaluation function is only a heuristic estimate.) Your function should also return the number of nodes expanded and the CPU time.

```
Procedure MinimaxAlphaBeta
Input: Position u, value α, value β
Output: Value at root

if (leaf(u)) return Eval(p)                    ;; No successor, return evaluation
if (max-node(u))                               ;; MAX node
   res ← α                                     ;; Initialize result value
   for each v ∈ Succ(u)                        ;; Traverse successor list
      val ← MinimaxAlphaBeta(v,res,β)          ;; Recursion for α
      res ← max{res,val}                       ;; Take maximal value
      if (res ≥ β)                             ;; Result exceeds threshold
         return res                            ;; Propagate value
else                                           ;; MIN node
   res ← β                                     ;; Initialize result value
   for each v ∈ Succ(u)                        ;; Traverse successor list
      val ← MinimaxAlphaBeta(v,α,res)          ;; Recursion for β
      res ← min{res,val}                       ;; Take minimal value
      if (res ≤ α)                             ;; Result exceeds threshold
         return res                            ;; Propagate value
return res                                     ;; Propagate value
```

Algorithm 12.4

Minimax game tree search with $\alpha\beta$-pruning.

4) Write a main function that loops through the following options until the user terminates the session with Option 5:

Option 0: This option allows the user to play with the computer. It asks if the human wants to play first. It then asks the depth *k* to which the tree will be expanded by the computer. It executes the game using the function *MinmaxAlphaBetaCutOff* , displaying the board after each turn. When the game ends, it displays the outcome of the game.

Option 1: takes as input a board *B* as a vector of vectors (such as [[1, 0, 0, 0], [0, 1, 0, 2], [0, 0, 0, 0], [0, 0, 0, 0]]. Then it calls the *Minimax* function and outputs the result.

Option 2: takes as input a board *B* and an integer *k*, calls *MinimaxAlphaBeta* function and reports the value of *B* using the evaluation function *e*. It also returns the best move for the root player.

Option 3: takes as input a board *B* and an integer *k*, calls both *MinmaxCutoff* and the *MinimaxAlphaBetaCutOff* function and reports the results.

Option 4: asks the user the enter a valid board. Then it asks the user to select 1 or 2. If the user selects 1 (2), the computer becomes player 2 (1). Next it asks the user to choose the cutoff depth (that represents the strength of the computer play). The game is then played between the user and the computer until the game ends. At each turn, the board is displayed. When the game ends, an appropriate message is displayed indicating who won.

Option 5: exit